

LLD Practice Platform

A focused, deliberate practice platform for Low-Level Design (LLD) and Object-Oriented Design (OOD). While AI feedback on LLD now exists (Hello Interview, algomaster.io, Low Level Design Mastery, lldproblems.com), it is per-session, ungrounded in the learner's own text, not pinned to a versioned rubric, and not aggregated across problems.

LLD Practice Platform solves this: learners choose a system design problem, author a structured design specification, receive grounded hybrid feedback with citations to specific paths in the learner's submission from deterministic heuristics and LLM evaluation, review score deltas over time, and systematically eliminate recurring architectural anti-patterns.

Key Features

- Structured Design Specification:** Normalizes design submissions into canonical sections (assumptions , entities , relationships , interfaces , tradeoffs , extensibility), avoiding ambiguous freeform prose while testing architectural intent rather than language boilerplate.
- Hybrid Evaluation Engine:**
 - Deterministic Rule Heuristics:** Zero-cost, instantaneous checks for structural anti-patterns (God classes, orphan entities, anemic domain models, unhandled extension axes, missing trade-offs).
 - LLM Architectural Judgment:** Deep qualitative evaluation of separation of concerns, single responsibility nuances, and trade-off justification using strict temperature-0 prompts validated via Zod schemas.
 - Confidence-Weighted Aggregation:** Evaluators run concurrently with per-evaluator timeouts; scores are merged via confidence weighting and degraded gracefully if an evaluator fails or is skipped.
- Grounded Evidence & Honest Citations:** Every finding carries an immutable EvidenceRef (quote pointing to the exact JSON path like entities[1].methods or explicit absence of a requirement). No fabricated quotes.
- State Machine & Failure Recovery:** Strict lifecycle (DRAFT → SUBMITTED → EVALUATING → EVALUATED / FAILED). Attempt records are persisted before evaluation starts. A startup stale-evaluation sweep automatically reclaims orphaned evaluations.
- Learning Loop & Longitudinal Analytics:** Tracks per-dimension score deltas between consecutive attempts and surfaces cross-attempt "Recurring Weaknesses" to guide targeted study.
- Clean Architecture & Information Hiding:** Inward-pointing clean architecture (domain layer has zero external dependencies). DTO layer strictly redacts internal rubric thresholds and expected concept lists from learner-facing API endpoints.

Architectural Highlights

Strict Clean Layering

```
src/
├─ domain/           # Pure domain models, state machine, value objects, domain errors
                      (ZERO external dependencies)
│   └─ models/       # Attempt aggregate, DesignSpec, Rubric, EvaluationReport,
Evidence, LearningLoop
│   └─ errors/       # Typed domain errors (ProblemNotFoundError,
ValidationFailedError, etc.)
├─ application/      # Orchestration services, evaluation engines, assemblers, learning
loop calculator
│   └─ evaluators/   # DeterministicEvaluator, LlmEvaluator, CompositeEvaluator,
EvaluationReportAssembler
│   └─ services/     # EvaluationService, LearningLoopService
├─ infrastructure/   # Concrete adapters, SQLite persistence, LLM client, seeds,
Express web server
│   └─ repositories/ # SqliteAttemptRepository, InMemoryAttemptRepository,
InMemoryProblemRepository
│   └─ formats/      # StructuredTextFormat (Factory pattern)
│   └─ llm/          # FakeLlmClient with offline determinism and simulated timeouts
```

```
|   |— seeds/          # 4 real-world seeded problems
|   |— web/           # Express app, DTO mappers, centralized error middleware, server
entrypoint
|— web/              # Client frontend: React 18, Vite, responsive CSS design system
```

Justified Design Patterns

1. **Strategy Pattern (Evaluator)**: Decouples evaluation engines (`DeterministicEvaluator` , `LlmEvaluator`) from orchestration logic, allowing new evaluators (e.g. human review or AST linters) to plug in seamlessly.
2. **Composite Pattern (CompositeEvaluator)**: Treats individual and composite evaluators uniformly, running members concurrently with per-evaluator timeouts and aggregating results into a single composite report.
3. **Repository Pattern (AttemptRepository , ProblemRepository)**: Isolates domain entities from database engines, allowing SQLite in production and blazing-fast in-memory repositories in unit tests.
4. **Factory Pattern (SubmissionFormat)**: Normalizes arbitrary inputs (structured text, and in future Mermaid diagrams or code) into canonical `DesignSpec` objects before reaching evaluators.

Prerequisites

- **Node.js**: Version 18.0.0 or higher
- **npm**: Version 9.0.0 or higher

Setup & Installation

Clone the repository and install dependencies:

```
git clone https://github.com/swarajladke/LLD-Practice-Platform.git
cd LLD-Practice-Platform
npm install
```

Running Locally

To run the complete platform locally:

1. Start the Backend API Server

In a terminal, start the Express backend with hot reloading (runs on port 3000):

```
npm run dev:server
```

The server will initialize the SQLite database (`lld_platform.db`), run the startup stale-evaluation recovery sweep, seed 4 core problems, and listen on `http://localhost:3000` .

2. Start the Frontend Application

In a second terminal, launch the Vite development server (runs on port 5173):

```
npm run dev
```

Open your browser to `http://localhost:5173` . The Vite dev server automatically proxies `/api` requests to the backend server at `http://localhost:3000` .

Running Tests

The test suite contains 64 unit and integration tests covering domain invariants, evaluation heuristics, concurrent composite execution, repository persistence, and API error mappings:

```
# Run all tests once
npm test

# Run tests in watch mode
npm run test:watch
```

Test Suite Structure

- `tests/architecture/DomainBoundary.test.ts` : Verifies zero external imports or forbidden node modules inside `src/domain/` .
- `tests/domain/AttemptStateMachine.test.ts` : Enforces state transitions, invariants, and rehydration safety.
- `tests/domain/EvidenceAndRubric.test.ts` : Validates quote/absence value objects and rubric normalization.
- `tests/application/DeterministicEvaluator.test.ts` : Tests all 8 positive and negative heuristic triggers (god classes, orphans, anemic models, missing abstractions, tradeoff checks).
- `tests/application/LlmAndCompositeEvaluator.test.ts` : Verifies pure stateless concurrent execution, simulated timeouts, skipped evaluators, and composite nesting.
- `tests/application/EvaluationAndLearningLoopService.test.ts` : Verifies idempotency safeguards, payload mismatch detection, stale recovery sweeps, and score delta calculations.
- `tests/infrastructure/AttemptRepository.test.ts` : Validates SQLite and in-memory idempotency key uniqueness and persistence.
- `tests/web/ApiEndpoints.test.ts` : Validates HTTP status code contracts (200, 201, 202, 400, 404, 409, 413, 422, 500), DTO redaction, and error mapping middleware.

Reviewer Walkthrough & Demo Script

Follow this step-by-step click-through to evaluate the platform end-to-end:

1. Select a Problem:

- On `http://localhost:5173` , the sidebar displays 4 seeded problems: *Parking Lot System*, *Elevator Dispatcher*, *Vending Machine*, and *Rate Limiter*.
- Click **Parking Lot System**.
- Notice the requirements, clarifying context, and extension axes displayed in the problem details pane. (Inspect the network response: rubric thresholds and expected concept arrays are strictly redacted by the DTO layer).

2. Load Starter Template:

- In the Submission Editor pane, click **Load Template**.
- A structured JSON template pre-populates with valid sections: `assumptions` , `entities` , `relationships` , `interfaces` , `tradeoffs` , and `extensibility` .

3. Submit Attempt:

- Click **Submit Design for Evaluation**.
- The UI generates a unique idempotency key and dispatches `POST /api/attempts` .
- The backend responds with `201 Created` and enqueues evaluation asynchronously.
- The UI polls `GET /api/attempts/:id` for lifecycle status (`SUBMITTED` → `EVALUATING`) and `GET /api/attempts/:id/report` (`202 Accepted` while evaluating).

4. Inspect the Evaluation Report:

- Once evaluation completes, the server returns `200 OK` with the full `EvaluationReport` .

- Review the **Overall Score** (0-10) and the **Evaluator Provenance** badges (`deterministic: completed` , `llm: completed`).
- Expand the **Dimension Breakdown** to see scores across all 8 rubric criteria:
 - Requirement Understanding
 - Class Responsibilities
 - Coupling & Cohesion
 - Encapsulation & Interfaces
 - Abstraction & Patterns
 - Extensibility & Change
 - Edge Cases & Testability
 - Explanation & Tradeoffs
- Click on findings to inspect honest citations (`quote` with target JSON path or explicit `absence` notice).

5. **Test Idempotency & Conflict Handling:**

- Click **Submit Design for Evaluation** again without modifying the JSON.
- The server detects the identical payload and replays the existing attempt (`200 OK` , same `attemptId`).
- Now modify a single character in the editor and click submit with the same key.
- The backend catches the collision and returns `409 Conflict` (`IdempotencyPayloadMismatchError`).

6. **View Attempt History & Longitudinal Learning Loop:**

- Make a second, improved submission for the same problem (e.g. break down a large entity or add an interface).
- In the **Attempt History** tab, observe the calculated score deltas (e.g., `+1.8` overall, with per-dimension gain/loss chips).
- Switch to the **Recurring Weaknesses** panel: any architectural flaw flagged in 2 or more attempts is aggregated with actionable recommendations.

Key Design Decisions & Tradeoffs

Decision	Alternative Considered	Rationale
Structured Spec (JSON)	Freeform prose / markdown	Freeform prose induces heavy LLM hallucination and prevents syntactic heuristics. Structured input provides reliable AST-like anchors for deterministic analysis.
Monolith In-Process Async	Celery / BullMQ / Redis	Monolithic simplicity meets take-home scope boundaries. A startup recovery sweep handles process crashes without adding external infrastructure dependencies.
Confidence-Weighted Merging	Winner-Takes-All scoring	Deterministic rules have near 1.0 confidence on syntax violations, while LLMs provide 0.8 confidence on design nuances. Merging preserves both findings with attribution.
DTO Redaction Layer	Exposing domain models directly	Prevents learners from inspecting browser DevTools to read <code>expectedConcepts</code> and game the evaluation heuristics.

For full details, read [docs/DESIGN.md](#) and [docs/RESEARCH.md](#).

AI Usage & Engineering Decision Log

This document records the collaborative pairing process between the engineer and AI during the design and implementation of the **LLD Practice Platform**. It documents critical architectural inflection points, what was proposed, what was accepted, and two significant defects where AI-generated code passed existing unit tests and was subsequently caught in architectural review.

Summary of Decisions

#	Topic	AI Proposal / Code	Outcome	Key Rationale
1	Persistence Layer	Introduce Prisma ORM or TypeORM with schema migrations	REJECTED	Preserved monolith simplicity; plain SQLite + explicit Repository pattern enables zero-dependency testing without ORM baggage.
2	Score Aggregation Strategy	Winner-Takes-All: LLM score completely overrides deterministic heuristics	REJECTED	Destroys objective syntactic signals. Adopted Confidence-Weighted Merging with full finding provenance.
3	Evaluation Concurrency	Deploy BullMQ or RabbitMQ with Redis distributed queue	REJECTED	Explicitly out-of-scope for 2-day monolith. Monolith in-process async with startup stale recovery sweep achieved resilience without external infra.
4	Evidence Representation	Introduce EvidenceRef discriminated union (quote vs absence)	ACCEPTED	Replaced fabricated quote strings like <code>Evidence.create('entities: []')</code> with honest absence notices.
5	Heuristic Finding Collection	Accumulate all findings per dimension and calculate score via <code>deriveScoreFromFindings</code>	ACCEPTED	Replaced early-exit heuristics with exhaustive feedback across all checks within a dimension.
6	Evaluator Concurrency State	Store execution state (<code>evaluatorsRun</code> , etc.) as instance variables on <code>CompositeEvaluator</code>	PASSED TESTS, CAUGHT IN REVIEW	Passed all sequential tests (64 tests across 8 suites); caught in review because concurrent runs on a shared instance cross-contaminate provenance. Refactored to pure stateless execution.
7	DTO & Information Hiding	Serialize domain models directly or place <code>toDto()</code> methods on domain entities	PASSED TESTS, CAUGHT IN REVIEW	Passed functional tests, but leaked rubric internals (<code>expectedConcepts</code> , <code>godClassMethodThreshold</code>) to the API. Refactored to an explicit DTO layer with redaction.
8	Competitive Research Positioning	AI initially framed positioning as "no structured LLD feedback exists"	REJECTED	Factually incorrect; research revealed four platforms shipping AI feedback. Positioning narrowed to grounding citations, rubric versioning, and cross-attempt aggregation.

Detailed Decision Logs

1. Persistence Layer: ORM (Prisma / TypeORM) vs. Repository Interface

- Context:** We needed persistent storage for `Attempt` records across server restarts, while supporting fast unit testing.
- AI Proposal:** Use Prisma ORM with a PostgreSQL/SQLite connection, generating Prisma client types and running SQL migrations.
- Decision:** REJECTED.
- Rationale:**
 - The assignment explicitly limits architectural scope to a domain-focused monolith. Introducing an ORM brings heavy binary dependencies (`prisma-client` , schema engines), slow migration steps, and leaky abstractions.
 - Instead, we defined an explicit `AttemptRepository` port in the domain layer and implemented two adapters:
 - `InMemoryAttemptRepository` for sub-millisecond, zero-dependency unit tests.
 - `SqliteAttemptRepository` using synchronous `better-sqlite3` for local persistence.
 - This preserves clean architecture and guarantees that testing the domain state machine requires zero database setup.

2. Evaluator Merging: Winner-Takes-All vs. Confidence-Weighted Findings

- **Context:** Both `DeterministicEvaluator` (syntactic rules) and `LlmEvaluator` (architectural judgment) evaluate overlapping rubric dimensions (e.g., `classResponsibilities`, `abstractionPatterns`).
 - **AI Proposal:** Use a "Winner-Takes-All" hierarchy where the LLM's score and commentary completely overwrite the deterministic rule outputs whenever the LLM succeeds.
 - **Decision: REJECTED.**
 - **Rationale:**
 - LLMs frequently suffer from sycophancy or overlook concrete syntactic violations (e.g. an entity having 8 methods or missing from relationships). Overwriting deterministic heuristics throws away objective, zero-cost truth.
 - Conversely, averaging bare numbers without context hides *why* a score was lowered.
 - **Adopted Solution:** We implemented **Confidence-Weighted Merging**: $\text{weight}_i = \max(0.1, \text{confidence}_i)$, $\text{Score} = \frac{\sum (\text{score}_i \times \text{weight}_i)}{\sum \text{weight}_i}$ Each finding retains its originating `evaluatorId` and cited `EvidenceRef`. The final report combines findings from both evaluators, giving the learner an undiluted view of both structural violations and nuanced architectural feedback.
-

3. Background Evaluation: Distributed Task Queue vs. Monolithic In-Process Async

- **Context:** LLM evaluations take between 2 and 15 seconds. Blocking the HTTP submission request (`POST /api/attempts`) would cause client timeouts, browser stalls, and dropped attempts if the connection breaks.
 - **AI Proposal:** Deploy BullMQ or RabbitMQ with Redis to manage an asynchronous background worker pool.
 - **Decision: REJECTED.**
 - **Rationale:**
 - The problem specification strictly mandates: *"Monolith is expected. Do NOT build microservices, Kubernetes, sharding, CDN, or queue infrastructure."*
 - Adding Redis and a separate worker runner adds operational complexity and violates the scope boundary.
 - **Adopted Solution:** We designed an **In-Process Asynchronous Pipeline**:
 1. `POST /api/attempts` persists the attempt in state `SUBMITTED` and immediately responds with `201 Created`.
 2. Evaluation runs asynchronously in the Node.js event loop via `void this.runEvaluationAsync(attemptId, problemId, spec)`.
 3. The client polls `GET /api/attempts/:id` for lifecycle status, and `GET /api/attempts/:id/report` for the report (`202 Accepted` while evaluating, `200 OK` on completion, `409 Conflict` on failure).
 4. To handle server crashes or restarts, we introduced a **Stale Evaluation Sweep** (`recoverStaleEvaluations()`) that runs on server startup, failing any attempt stuck in `EVALUATING` past a configurable deadline.
-

4. Evidence Representation: `EvidenceRef` Union Replacing Fabricated Quotes

- **Context:** Findings need to cite concrete evidence from the learner's submission. When a required entity, relationship, or section was missing, the earlier code fabricated fake quotes such as `Evidence.create('entities: []', 'spec.entities')` or `Evidence.create('No extensibility details', 'extensibility')`.
- **AI Proposal:** Introduce a discriminated union:

```
export type EvidenceRef =
  | { readonly kind: 'quote'; readonly evidence: Evidence }
  | { readonly kind: 'absence'; readonly expectedPath: string; readonly note:
    string };
```

- **Decision: ACCEPTED.**
- **Rationale:**
 - The domain should never invent quotations that do not exist in the learner's text.

- The `EvidenceRef` union models reality honestly: positive findings quote the learner's actual text with exact JSON path coordinates (e.g. `entities[0].methods[2]`), while missing sections or unfulfilled rubric requirements are clearly identified as `absence` references with an explanatory note.

5. Heuristic Finding Collection: Exhaustive Findings via `deriveScoreFromFindings`

- **Context:** Initial heuristic implementations exited early on the first detected issue (e.g. bailing out immediately if an entity lacked an interface, without checking for anemic attributes or God-class method counts).
- **AI Proposal:** Refactor all deterministic heuristics to collect all findings across the dimension, and derive the dimension score deterministically using a unified scoring function `deriveScoreFromFindings(findings, baseScore)` .
- **Decision: ACCEPTED.**
- **Rationale:**
 - Early-exit heuristics force the learner into a frustrating "whack-a-mole" cycle: they fix one issue, re-submit, and only then discover three other violations on the same dimension.
 - Accumulating all findings provides complete diagnostic clarity in a single report.

6. Evaluator Concurrency State: Mutable Instance State Caught in Review

- **Context:** `CompositeEvaluator` runs registered sub-evaluators and tracks provenance (`evaluatorsRun` , `evaluatorsFailed` , `evaluatorsSkipped`).
- **AI-Generated Code:** The AI originally implemented provenance tracking using mutable private class fields:

```
class CompositeEvaluator {
  private evaluatorsRun: string[] = [];
  private evaluatorsFailed: string[] = [];
  // ...
  public getProvenance() { return { ... }; }
}
```

- **Why Existing Tests Missed It:**
 - The test suite had 64 unit tests across 8 suites. Every single test executed evaluations sequentially on a freshly instantiated `CompositeEvaluator` . Because no test ran evaluations concurrently on the same instance, all tests passed green with zero errors.
- **Review Finding & Resolution:**
 - In a real Node.js web server, `CompositeEvaluator` is instantiated as a singleton shared across concurrent HTTP requests.
 - If two learners submit designs concurrently, their evaluations share the instance. If learner A's LLM call fails while learner B's succeeds, `this.evaluatorsFailed` is mutated in place, causing learner B's report to be falsely marked as `degraded: true` !
 - **Fix:** Removed all instance state. `evaluate(ctx)` was made a pure, stateless function returning:

```
{
  readonly results: readonly DimensionResult[];
  readonly provenance: {
    readonly evaluatorsRun: readonly string[];
    readonly evaluatorsFailed: readonly FailedEvaluatorInfo[];
    readonly evaluatorsSkipped: readonly string[];
  };
}
```

- Added an explicit concurrency test in `tests/application/LlmAndCompositeEvaluator.test.ts` executing two evaluations simultaneously on one `CompositeEvaluator` instance (one failing, one succeeding) to prove complete isolation.

7. API Boundary & Information Hiding: Domain Serialization Caught in Review

- **Context:** Problem requirements and rubric definitions must be served to the frontend practice UI.
- **AI-Generated Code:** The AI initially returned raw `Problem` domain models directly from Express route handlers or attached `toDto()` methods directly to domain classes.
- **Why Existing Tests Missed It:**
 - The API tests checked that `GET /api/problems/:id` returned status 200 and that `body.id` matched. The tests did not assert the *absence* of internal fields.
- **Review Finding & Resolution:**
 - Exposing the raw `Problem` domain entity leaked internal evaluation rubric secrets: `expectedConcepts` (e.g. `['Spot', 'Vehicle', 'Ticket']`), `minEntities`, `minTradeoffs`, and `godClassMethodThreshold`.
 - Any learner could open browser DevTools, inspect the `/api/problems/:id` response, and trivially game the evaluation heuristics.
 - Furthermore, having `toDto()` on domain models coupled the domain layer to HTTP wire representations.
 - **Fix:** Created an explicit DTO mapping layer in `src/infrastructure/web/dto/index.ts`. `ProblemDto` exposes public requirements, clarifying context, and extension axes, while completely omitting all internal rubric thresholds and concept keywords.
 - Added an integration test in `tests/web/ApiEndpoints.test.ts` asserting that `GET /api/problems/:id` contains none of the rubric internal keys.

8. Competitive Research: AI-Gathered, Human-Filtered

- **Context:** Mapping the competitive landscape of existing Low-Level Design preparation platforms to position our product.
- **AI Proposal & Initial Framing:** The AI ran the initial web research and drafted the competitive comparison table and market gap analysis. It initially framed the product positioning around the premise that "no automated or structured LLD feedback exists in the market."
- **Review Finding & Human Verification:**
 - The human engineer verified the research rows against available public marketing and free-tier pages. (Note: Premium feedback internals for Hello Interview, algomaster.io, Low Level Design Mastery, and lldproblems.com are behind paywalls and explicitly marked as unverified in `docs/RESEARCH.md`).
 - The AI's initial framing ("no structured LLD feedback exists") was **REJECTED** as factually wrong after the research surfaced that four active competitors are already shipping automated or LLM-driven LLD feedback experiences.
- **Decision & Refined Positioning:**
 - The positioning was narrowed to what actually differentiates the platform: AI feedback on LLD now exists, but it is per-session, ungrounded in the learner's own text with citations to specific paths in the learner's submission, not pinned to a versioned rubric, and not aggregated across problems into a longitudinal learning loop.

Reflections on AI Collaboration

The AI was exceptionally effective at:

1. Quickly generating exhaustive test suites across heuristic positive/negative boundaries.
2. Formulating TypeScript discriminated unions and exhaustiveness checking for state transitions.
3. Rapidly scaffolding CSS styling tokens and React state management.

Human architectural judgment was required to:

1. Enforce strict scope boundaries and reject unnecessary distributed infrastructure (Redis, BullMQ, Prisma).
2. Catch subtle concurrency bugs stemming from mutable class instances that passed all sequential unit tests.
3. Protect domain invariants from leaky abstractions and cheating vectors (DTO redaction).